

PART XXV**ADVANCED PROBLEM SOLVING**

Hard-problem thinking, upsolving, pattern recognition, and proving you're right

Every Part before this one taught you tools. This Part is about the thing that actually separates someone who knows every algorithm in this book from someone who can solve a problem they have never seen before: the process that runs in your head between reading a statement and writing the first line of code. Nothing here is a new data structure. Everything here is a habit — how to attack an unfamiliar problem, how to recognize a pattern that isn't labeled, how to learn from a problem you failed to solve, and how to know your solution is correct before the judge tells you.

Contents of Part XXV

- 1. How to Think Before Coding
- 2. Brute Force → Optimization as a Repeatable Cycle
- 3. Hard Problem Training: The Full Pipeline
- 4. Pattern Recognition Without Labels
- 5. Proof and Correctness
- 6. Up-Solving: Learning From Failure
- 7. Edge Cases and Systematic Testing
- 8. Common Competitive Programming Mistakes
- 9. Unlabeled Pattern Drills
- Part XXV Revision, Cheat Sheet, and Interview Questions

Chapter 1 — How to Think Before Coding

The single most common mistake at every rating level below about 1600 is the same one: reading a problem and immediately starting to type. This chapter is the antidote — a fifteen-step process that front-loads thinking so that when you do start coding, you're implementing a decision you've already made rather than discovering the solution by trial and error inside the editor.

1.1 The fifteen steps

- 1 Understand the statement. Re-read it. Restate the task in your own words in one sentence.
- 2 Identify input and output. Exactly what is given, exactly what must be printed, in what format.
- 3 Study the constraints. Before anything else — they tell you the intended complexity (Chapter 2).
- 4 Create your own examples. Especially tiny ones: $n = 1$, $n = 2$, all-equal, already-sorted.
- 5 Find a brute force. The dumbest correct solution you can describe, even if hopelessly slow.
- 6 Calculate the brute force's complexity. Concretely, against the actual constraints.
- 7 Find the bottleneck. Which specific step makes it too slow? Not “it's slow” — which loop, which repeated computation.
- 8 Look for patterns. Compute small cases by hand and look at the sequence of answers.
- 9 Choose a data structure. What operation does the bottleneck need done faster?
- 10 Choose an algorithm. Guided by the pattern and the target complexity, not by what you used last.
- 11 Prove or justify the approach. At minimum, convince yourself why it can't be wrong (Chapter 5).
- 12 Code it. Only now.

- 13 Test edge cases. The tiny examples from step 4, plus the systematic list in Chapter 7.
- 14 Analyze the final complexity. Confirm it actually fits the constraints.
- 15 Optimize if needed. Only if step 14 says you must.

♦ Why step 3 comes before step 5

Reading constraints before designing anything is the habit that separates efficient practice from wasted hours. If $n \leq 10^5$, you already know an $O(n^2)$ approach is dead before you spend twenty minutes designing one. The constraints are the problem setter telling you the intended complexity — they are not decoration.

1.2 Constraints → intended complexity, revisited

Constraint on n	Likely intended complexity	Technique family
$n \leq 10$	$O(n!)$ or exhaustive search	Full permutation enumeration, backtracking
$n \leq 20$	$O(2^n)$	Subset enumeration, bitmask DP
$n \leq 40$	$O(2^{(n/2)})$	Meet in the middle
$n \leq 100$	$O(n^3)$	Floyd-Warshall, interval DP
$n \leq 1,000$	$O(n^2)$	2D DP, all-pairs loops
$n \leq 100,000$	$O(n \log n)$	Sorting, binary search, heap, segment tree
$n \leq 1,000,000$	$O(n)$ or $O(n \log n)$	Two pointers, prefix sums, sliding window, hashing
$n \geq 10^9$ (as a VALUE, not a count)	$O(\log n)$ or $O(\sqrt{n})$	Binary exponentiation, math, digit DP

Read that last row carefully — it's a distinction that trips people up constantly. A constraint of 10^9 on the number of elements would be absurd; a constraint of 10^9 on a single value (an amount, a coordinate, a target) is routine and usually signals math, binary search on the answer, or digit DP rather than any kind of iteration over the range.

⚠ These are guidelines, not laws

A judge typically runs 10^8 to 10^9 simple operations per second, but constant factors matter enormously: an $O(n \log n)$ solution with heavy per-element work can lose to an $O(n^2)$ solution with trivial per-element work at moderate n . Use the table to narrow your search, then sanity-check against the actual operations your solution performs.

Chapter 2 — Brute Force → Optimization as a Repeatable Cycle

This book has repeated one cycle in nearly every Part. Here it is stated explicitly, because it is the actual engine of competitive problem solving:

- 1 Naive — write (or at least fully describe) the obviously-correct slow solution.
- 2 Why it's slow — identify the exact bottleneck operation and how many times it runs.
- 3 Observation — find a property of the problem that makes some of that work redundant.
- 4 Improvement — apply a data structure or technique that exploits that property.
- 5 Optimal — confirm the new complexity fits, and that correctness survived the change.

2.1 A worked example of the full cycle

Problem: given an array of n integers and a target S , count pairs (i, j) with $i < j$ and $a[i] + a[j] == S$. Constraint: $n \leq 10^5$.

Stage	Content
Naive	Two nested loops over all pairs, checking each sum — $O(n^2)$
Why it's slow	$n^2 = 10^{10}$ pair checks at $n = 10^5$, roughly $100\times$ too slow for a 1-second limit

Stage	Content
Observation	For a fixed $a[i]$, the only partner that works is exactly $S - a[i]$ — we don't need to SEARCH for it, we can LOOK IT UP
Improvement	A hash map from value $\rightarrow$ count of occurrences turns the inner loop into an $O(1)$ lookup
Optimal	One pass: for each $a[i]$, add the count of $(S - a[i])$ seen so far, then record $a[i]$ — $O(n)$ time, $O(n)$ space
<pre> long long countPairs(vector<int> &a, int S) { unordered_map<int,int> seen; // value -> how many times seen SO FAR long long count = 0; for (int x : a) { count += seen[S - x]; // every earlier occurrence of S-x forms a valid pair seen[x]++; // record x AFTER counting, to enforce i < j } return count; } </pre>	

Notice where the correctness argument lives: recording x after counting is what guarantees $i < j$ and prevents an element pairing with itself. That single line ordering is the kind of detail that step 11 of Chapter 1 (“prove or justify”) is meant to catch — before submission, not after a wrong-answer verdict.

✦ The habit this builds

Never skip the naive step, even when you think you already see the optimal solution. The naive version gives you two things you cannot get any other way: a definitively correct reference to test against (Chapter 7's stress testing), and a concrete bottleneck to attack rather than a vague feeling that the problem is hard.

Chapter 3 — Hard Problem Training: The Full Pipeline

For a genuinely hard problem (roughly Codeforces Div. 2 D and above, or LeetCode Hard), the jump from “problem” to “solution” is never

direct. The pipeline below is what that jump actually looks like when written out.

Problem → constraints → brute force → bottleneck → observation → mathematical/algorithmic insight → data structure → optimization → final algorithm → proof → implementation → complexity check

3.1 Techniques for generating the observation

“Find the observation” is the step people get stuck on, and it's the one that feels like it requires talent. It mostly requires a checklist. When stuck, work through these deliberately:

- Compute small cases by hand. $n = 1, 2, 3, 4, 5$. Write down the answers in a row. Look for a pattern, a recurrence, or a formula.
- Ask what's invariant. What quantity never changes as the process runs? Invariants are the backbone of most constructive and greedy proofs.
- Ask what's monotonic. Does some quantity only ever increase, or only decrease? Monotonicity is what makes binary search on the answer legal.
- Reverse the problem. Instead of counting things that satisfy a property, count the total and subtract those that don't (complementary counting).
- Fix one variable. If the answer depends on two things, fix one and see whether the problem becomes easy in the other.
- Look at the problem from the end. Process in reverse order; many problems that are hard forward are easy backward.
- Ask what the constraint is FOR. An unusual constraint (“ $k \leq 20$ ”, “values are distinct”, “the tree is a path”) is almost always load-bearing — the intended solution uses it.

♦ The most underused of these

“Ask what the constraint is for.” Problem setters do not include an odd, specific constraint by accident. If a problem says $a_i \leq 100$ while $n \leq 10^5$, the intended solution almost certainly involves a frequency array or DP over the tiny value range — the constraint is a hint, delivered in plain sight.

3.2 What to do when genuinely stuck

Time stuck	What to do
0–10 min	Keep thinking. Re-read the statement — misreading is the most common cause of early blockage.
10–25 min	Work the observation checklist above explicitly, one item at a time, writing things down.
25–40 min	Solve a simplified version (smaller constraints, one dimension instead of two, a special case). Often the general solution follows.
40+ min (practice)	Stop and up-solve deliberately (Chapter 6) — read the editorial, but stop reading the moment you have the key idea and finish it yourself.
40+ min (contest)	Move to another problem. Return only if time remains; a solved easier problem is worth more than an unsolved harder one.

Chapter 4 — Pattern Recognition Without Labels

In a contest, nobody tells you “this is a sliding window problem.” The entire skill is producing that label yourself, from the statement. This chapter gives the clue-to-pattern mapping in the direction you actually need it: from phrasing to technique.

4.1 The recognition table

Clue in the statement	Likely pattern
“contiguous subarray” + a constraint on its content	Sliding window (variable or fixed size)
“sum of a range”, repeated queries, no updates	Prefix sums
“range update” then a final read	Difference array
“range query” AND “update” together	Fenwick tree / segment tree
Sorted array + “find a pair/triplet”	Two pointers
“minimum possible maximum” / “maximum possible minimum”	Binary search on the answer
“minimum number of steps” in an unweighted graph or grid	BFS
“shortest path” with non-negative weights	Dijkstra
“are these connected” / repeated merging of groups	DSU
“prerequisites” / “ordering with dependencies”	Topological sort
“k largest/smallest”, or repeatedly take the best item	Heap
“next greater/smaller element”, or histogram-shaped	Monotonic stack
“intervals” + maximize count of non-overlapping	Greedy, sort by end time
“intervals” + count overlaps at a point	Sweep line
“maximize/minimize” + choices compound + n is moderate	Dynamic programming
$n \leq 20$ + “which subset” matters	Bitmask DP
“how many numbers in [L, R] have digit property X”	Digit DP
“count prefixes” / “words starting with”	Trie

Clue in the statement	Likely pattern
"substring occurs in" / pattern matching	KMP, Z-algorithm, or hashing

4.2 Why this table is dangerous if memorized carelessly

The table above is a set of priors, not rules. A problem that mentions intervals may not be a greedy problem; a problem with $n \leq 20$ may be solvable in polynomial time and the small constraint is a red herring. Use the table to generate a short list of candidates to test against the problem's actual structure, then verify one of them genuinely fits. Applying a pattern because a keyword matched, without checking the structure, is exactly how confidently wrong submissions happen.

✦ How to practice this properly

Solve problems WITHOUT looking at their tags or the topic heading of the section they came from. If you know in advance that you're in the "DP chapter," you aren't practicing recognition — you're practicing implementation of a pattern already handed to you. Mixed, untagged problem sets are the only way to train the skill that contests actually test. Chapter 9 provides a set to start with.

Chapter 5 — Proof and Correctness

You do not need formal mathematics to be confident in a solution. You need to be able to answer one question: what would have to be true for this to be wrong? — and then check whether it can be.

5.1 Proof techniques by algorithm family

Algorithm family	The argument that justifies it
Binary search	The predicate is monotonic — false...false, true...true with exactly one boundary. Verify the monotonicity; the search is then trivially correct.
Greedy	Exchange argument — any optimal solution can be transformed into the greedy one without getting worse.

Algorithm family	The argument that justifies it
Dynamic programming	The recurrence covers every case exhaustively and each state depends only on strictly smaller states (no circularity).
BFS shortest path	Vertices are dequeued in non-decreasing distance order, so the first time a vertex is reached is via a shortest path.
Dijkstra	Finalizing the minimum-distance unvisited vertex is safe because all remaining edges are non-negative — nothing can improve it later.
DSU / Kruskal	The cut property — the cheapest edge crossing any partition belongs to some MST.
Topological sort	A vertex is output only when all its prerequisites already are; if some vertices never qualify, a cycle exists.

5.2 The three questions to ask before submitting

- 1 Is my approach correct in general, or only on the examples I tried? Can I state WHY it can't fail, in one or two sentences?
- 2 Does my complexity actually fit? Compute it concretely against the maximum constraints — not “it's about $n \log n$,” but “ $10^5 \times 17 \approx 1.7 \times 10^6$ operations, comfortably fine.”
- 3 What is the smallest input that could break this? Run through the edge-case list from Chapter 7 mentally.

⚠ The greedy-specific warning

Greedy is the family where “it worked on my examples” most reliably misleads. If your solution is greedy and you cannot produce an exchange argument, actively spend two minutes trying to BREAK it with a small adversarial case before submitting. This single habit prevents a large fraction of wrong-answer verdicts at intermediate ratings.

Chapter 6 — Up-Solving: Learning From Failure

In one sentence: up-solving is deliberately returning to a problem you failed to solve during a contest and solving it properly afterward — and it is, by a wide margin, the highest-value practice activity available to you.

The reason is simple: problems you solve during a contest teach you almost nothing, because you already knew how. Problems you failed teach you exactly the thing you didn't know. Contestants who plateau are almost always the ones who compete regularly and never up-solve.

6.1 The up-solving protocol

- 1 Try again cold, first. Before any editorial, spend 20–30 more minutes on the problem with contest pressure removed. A surprising fraction become solvable.
- 2 Read the editorial only until the key idea. The moment you hit the observation you were missing, STOP reading and go finish the solution yourself.
- 3 Implement it yourself, from scratch. Never copy the editorial's code. If you can't implement it without the code in front of you, you haven't learned it.
- 4 Write down what you missed — specifically. Not “it was a DP problem” but “I didn't notice that processing in reverse order removes the dependency.”
- 5 Find and solve two similar problems within the next few days, to convert the one-off insight into a recognized pattern.

6.2 The up-solving log

Keep a running log of every up-solved problem in a simple table. This becomes, over months, the single most valuable revision document you own — because it is a record of your specific, personal blind spots rather than a generic topic list.

Column	What goes in it
Problem	Link or identifier

Column	What goes in it
What I tried	The approach that failed, and why it failed
The observation I missed	Stated in one concrete sentence
Concept it teaches	The transferable pattern, if any
Similar problems solved since	Proof you converted it into a recognized pattern

✦ **Review your log, don't just write it**

Re-read the “observation I missed” column every few weeks. Repeated entries reveal a systematic gap (“I consistently fail to consider processing in reverse”) that no topic-based study plan would ever surface.

Chapter 7 — Edge Cases and Systematic Testing

7.1 The standard edge-case checklist

- $n = 0$ and $n = 1$ — does the code even run? Do loops with $n-1$ bounds break?
- Empty array, empty string, empty graph
- All elements equal
- Already sorted, and reverse sorted
- Duplicates present (especially when the logic assumed distinctness)
- Negative numbers, and zero specifically
- Maximum constraint values — does anything overflow int?
- A disconnected graph (BFS/DFS from one node misses components)
- A graph that is a single long chain (recursion depth)
- A cyclic graph where the algorithm assumed a DAG
- Single-character strings, and strings of all identical characters

7.2 Stress testing

In one sentence: write a random-input generator and compare your fast solution against your brute force on thousands of small random inputs until they disagree — at which point you have a minimal failing case, handed to you automatically.

```
#!/bin/bash
# stress.sh -- compare fast solution against brute force on
# random small inputs
for i in $(seq 1 1000); do
    ./gen $i > input.txt # generator, seeded by i, produces SMALL
    random input
    ./fast < input.txt > out_fast.txt
    ./brute < input.txt > out_brute.txt
    if ! diff -q out_fast.txt out_brute.txt > /dev/null; then
        echo "Mismatch found on test $i"
        cat input.txt
        break
    fi
done
echo "Done"
```

The generator must produce small inputs (n around 5–8, values in a tiny range) for two reasons: the brute force must finish quickly, and a failing case small enough to trace by hand is far more useful than a failing case with $n = 1000$. This technique is the reason Chapter 2 insists on never skipping the brute force — it isn't just a stepping stone to the optimal solution, it's your test oracle.

♦ When to reach for stress testing

Whenever you're confident your approach is correct but you're getting a wrong answer on a test you can't see. Manually guessing at edge cases can take an hour; stress testing typically finds the counterexample in under a minute, and hands you a minimal reproduction.

Chapter 8 — Common Competitive Programming Mistakes

Mistake	Symptom / Fix
Integer overflow	Wrong answer on large tests — use long long for any sum/product that can exceed $\sim 2 \times 10^9$
Wrong complexity for the constraints	TLE — compute the operation count concretely before coding, not after
Off-by-one in loop or binary search bounds	Wrong answer on boundary tests — dry run on $n=1$ and $n=2$ explicitly
Incorrect binary search (infinite loop or wrong boundary)	TLE or wrong answer — verify the predicate is monotonic and the loop invariant shrinks the range every iteration
Unhandled duplicates	Wrong answer — check whether the logic assumed distinct values
Incorrect greedy assumption	Wrong answer — no exchange argument means hunt for a counterexample first
DP state missing a dimension	Wrong answer — symptom: the same 'state' seems to need two different answers
Graph indexing mismatch (0-indexed vs 1-indexed)	Runtime error or silently wrong — pick one convention per solution and size arrays as $n+1$
Visited array not reset between test cases	Wrong answer only on multi-test-case inputs — clear all global state at the top of each test case
Recursion depth exceeded	Segmentation fault — convert to iterative DFS for large V , or for chain-shaped graphs
Iterator invalidation while modifying a container	Undefined behavior — don't erase from a container you're iterating without using the returned iterator
Negative modulo	Wrong answer — use $((a \% m) + m) \% m$ when a can be negative
Floating-point precision comparison	Wrong answer — compare with an epsilon, or restructure to use integers entirely
Missing fast I/O on large input	TLE — add <code>ios::sync_with_stdio(false); cin.tie(nullptr);</code>

♦ Build a personal version of this table

The generic list above is a starting point. The genuinely useful version is the one you build from your own up-solving log (Chapter 6) — because the three or four mistakes YOU actually make repeatedly matter far more than the eleven you never make.

Chapter 9 — Unlabeled Pattern Drills

The problems below are deliberately given without their topic. For each one: read it, spend at least five minutes identifying the pattern and designing an approach BEFORE looking at the revealed pattern underneath. The point is not to solve them all right now — it's to practice producing the label yourself.

Drill Set A

Problem 25.1 — Longest substring with at most K distinct characters [Level 3]

Try yourself first: Design an approach before reading on.

Hint 1: The substring must be contiguous, and a constraint limits what it can contain.

Key observation: As the right end expands, the left end only ever needs to move right — it never needs to go back.

Underlying pattern: Sliding window (variable size) with a frequency map.

Complexity: $O(n)$.

Problem 25.2 — Given ropes of various lengths, connect all into one at minimum total cost, where connecting two ropes costs the sum of their lengths [Level 2]

Try yourself first: What determines the total cost?

Key observation: A rope's length is re-counted once for every merge it participates in, so the cheapest ropes should be merged earliest.

Underlying pattern: Greedy via a min-heap (the Huffman pattern).

Complexity: $O(n \log n)$.

Problem 25.3 — Find the minimum capacity of a ship that can deliver all packages within D days, shipping packages in their given order [Level 4]

Try yourself first: The answer is a number — what's the search space?

Key observation: If capacity C works, every capacity greater than C also works — the feasibility predicate is monotonic.

Underlying pattern: Binary search on the answer, with a greedy feasibility check.

Complexity: $O(n \log(\text{sum of weights}))$.

Drill Set B

Problem 25.4 — Given a list of accounts each with a name and emails, merge accounts that share any email [Level 4]

Try yourself first: What relationship connects two accounts?

Key observation: “Shares an email” is a merging relation that is transitive — A shares with B, B shares with C means all three are one account.

Underlying pattern: DSU (union by shared email).

Complexity: $O(\text{total emails} \times \alpha)$.

Problem 25.5 — Count how many numbers between L and R have no two adjacent equal digits [Level 5]

Try yourself first: Note the size of the range before deciding anything.

Key observation: The range can be up to 10^{18} — far too large to iterate. The property depends only on the digits themselves.

Underlying pattern: Digit DP, with the previous digit carried in the state.

Complexity: $O(\text{digits} \times 10 \times 2)$.

Problem 25.6 — For each day, find how many days you'd have to wait until a warmer temperature [Level 3]

Try yourself first: The naive solution is obvious — what makes it slow, and what's redundant about it?

Key observation: Once a warmer day is found for an earlier day, that earlier day is answered forever — pending days form a decreasing sequence.

Underlying pattern: Monotonic stack.

Complexity: $O(n)$ amortized.

Part XXV — Master Revision

Must Remember

- Read constraints BEFORE designing an approach — they tell you the intended complexity and kill dead-end designs for free.
- Never skip the brute force: it's both the starting point for optimization and your test oracle for stress testing.
- The optimization cycle is always the same: naive → why it's slow → observation → improvement → optimal.
- When stuck on the observation, work the checklist deliberately: small cases, invariants, monotonicity, reverse the problem, fix one variable, process backward, ask what the odd constraint is for.
- Pattern recognition means producing the label from the statement — practice on UNTAGGED problems, or you're only practicing implementation.
- The recognition table gives priors, not rules — always verify the structure actually fits before committing.
- Greedy is the family where 'it worked on my examples' misleads most — demand an exchange argument or actively hunt a counterexample.
- Up-solving is the highest-value practice activity: retry cold, read only to the key idea, implement yourself, log what you missed, then solve two similar problems.
- Stress testing finds counterexamples in seconds that manual guessing can take an hour to find.
- Your personal mistake list, built from your own up-solving log, matters more than any generic one.

Interview-Style Questions

- 1 Why should you read the constraints before designing an algorithm, rather than after?
- 2 Why is writing the brute-force solution valuable even when you already see the optimal approach?

- 3 What makes binary search on the answer valid, and how do you check that condition?
- 4 Why does practicing on tagged/topic-grouped problems fail to train pattern recognition?
- 5 Why is up-solving more valuable practice than solving additional problems you can already solve?
- 6 How does stress testing find a failing case, and why must the generated inputs be small?
- 7 Why is a greedy solution that passes several hand-tried examples still risky to submit?

Cheat Sheet — Part XXV

```
// THE PROCESS (before typing any code)
// 1 Understand 2 I/O 3 CONSTRAINTS 4 Examples 5 Brute force
// 6 Its complexity 7 Bottleneck 8 Pattern 9 Data structure 10
Algorithm
// 11 Justify 12 Code 13 Edge cases 14 Final complexity 15
Optimize if needed

// THE CYCLE
// Naive -> Why slow -> Observation -> Improvement -> Optimal

// WHEN STUCK ON THE OBSERVATION
// small cases by hand | invariants | monotonicity | reverse the
problem
// fix one variable | process backward | what is the odd
constraint FOR?

// BEFORE SUBMITTING
// 1 Why can this not be wrong (one sentence)?
// 2 Concrete op count vs. constraints (not 'about n log n')
// 3 Smallest input that could break it?

// WHEN WRONG-ANSWER ON A HIDDEN TEST
// stress test: gen (small random) + brute + fast + diff in a loop

// AFTER EVERY CONTEST
// up-solve: retry cold -> editorial to the key idea ONLY ->
implement yourself
// -> log the observation you missed -> solve 2 similar problems
```

Where this Part sits

The Master Roadmap lists Advanced Problem Solving as Phase 24 — the final phase of the learning path — and it is deliberately not a topic you finish. Every Part of this book feeds into it: the data structures give you options, the algorithms give you tools, and this Part is the process that decides which to reach for on a problem nobody has labeled for you. Revisit this Part periodically, not once.